



GUIA DIDÁTICO COMPLETO

Open RAN sem mistério

Kubernetes, Nephio, 5G Core, RIC, xApps e o laboratório multi-site da UnB.

OBJETIVO Ensinar do servidor físico ao KPI no Grafana, documentar o que foi construído e permitir que alunos reproduzam a PoC e criem novos experimentos.

NMI + CIC | PoC acadêmica | versão 1.0 | 1º de setembro de 2026

Repositório: github.com/lucasrodri/oran-stack

0 Como usar este guia

Este documento foi escrito para alunos e professores que ainda não dominam Kubernetes, telecom 5G ou Nephio. Ele não pressupõe que a pessoa conheça a sopa de letrinhas.

LEITURA EM DUAS VELOCIDADES Para uma visão geral, leia os capítulos 1 a 8 e 22 a 27. Para reproduzir, siga os capítulos 28 a 42 com o laboratório já provisionado.

O que você deve conseguir explicar ao final

- por que servidor físico, VM, nó, namespace, pod e contêiner não são sinônimos;
- por que Kubernetes e Nephio são complementares, não concorrentes;
- como UE, RAN, Open5GS, Near-RT RIC, xApp e Grafana formam um experimento;
- como um código Python vira imagem, pacote, variante, Deployment e métrica;
- onde cada componente está implantado no NMI e no CIC;
- quais limitações da PoC são intencionais e quais viram temas de pesquisa.

Partes do documento

Parte	Capítulos	Pergunta respondida
I · Fundamentos	1–8	o que é cada tecnologia?
II · Laboratório	9–15	onde tudo roda?
III · Repositório	16–21	onde está cada artefato?
IV · Fluxos	22–27	como a PoC funciona de ponta a ponta?
V · Reprodução	28–34	como validar e operar com segurança?
VI · Nova xApp	35–42	como criar um experimento próprio?
Apêndices	A–E	quais comandos e termos consultar?

ESCOPO É uma PoC acadêmica. Não buscamos alta disponibilidade, SLA ou backups de produção; buscamos transparência, repetibilidade e evidência científica.

1 Uma visão em seis camadas

O laboratório fica simples quando cada responsabilidade tem uma camada e um dono.

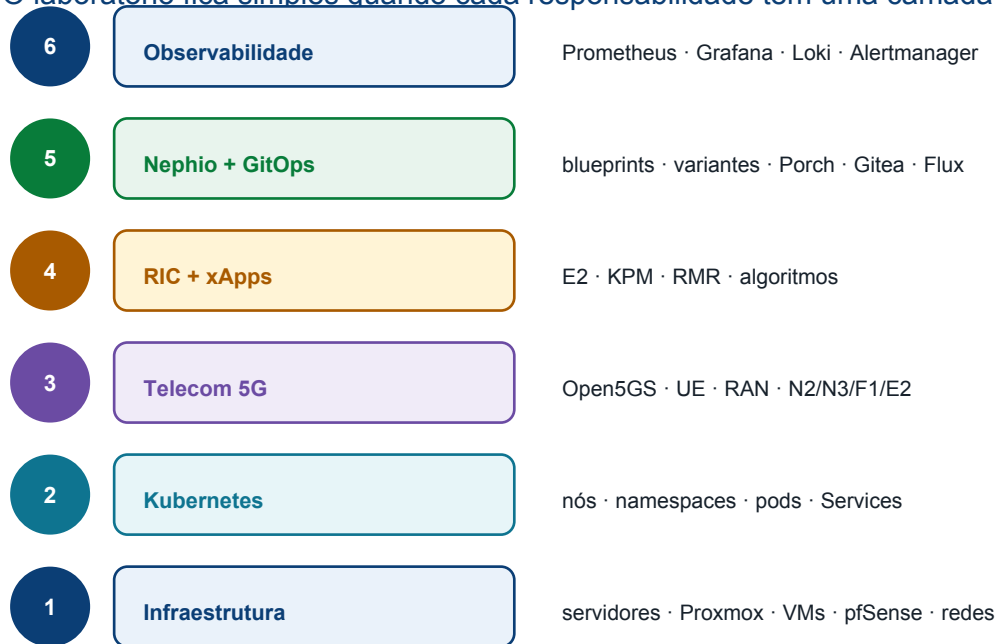


Figura 1 · As seis camadas da PoC, da infraestrutura à observabilidade.

REGRA MENTAL As camadas cooperam, mas uma não substitui a outra. O erro mais comum é pedir ao Kubernetes uma função do Nephio ou atribuir ao Nephio o tráfego 5G.

Uma frase por camada

- Infraestrutura fornece CPU, RAM, disco e redes.
- Kubernetes executa e reconcilia workloads em contêineres.
- Telecom 5G registra o UE e transporta os dados do usuário.
- RIC + xApps recebe telemetria da RAN e executa algoritmos.
- Nephio + GitOps organiza e entrega configuração por site.
- Observabilidade transforma estado, métricas e logs em evidência.

2 Do servidor ao pod

O caminho físico ajuda a entender onde um processo realmente está rodando.

Nível	Definição	Exemplo
Servidor físico	máquina real com CPU, RAM, NIC e disco	Dell ou Ampere
Hipervisor	software que hospeda VMs	Proxmox VE
VM	computador virtual com SO próprio	oran-k8s-01
Nó Kubernetes	máquina participante do cluster	control plane ou worker
Pod	menor unidade executável do Kubernetes	pod do AMF ou da xApp
Contêiner	processo empacotado com dependências	imagem oran-xapps

EXEMPLO REAL Proxmox002 hospeda a VM oran-k8s-01. Essa VM é um nó Kubernetes. Nela rodam pods do Open5GS, da RAN e do RIC base.

Um ou dez servidores?

A lógica de comunicação entre pods é a mesma. Mais máquinas aumentam capacidade, permitem distribuição de falhas e tornam o experimento multi-host, mas não mudam o modelo declarativo do Kubernetes.

POC, NÃO PRODUÇÃO Nesta bancada, distribuir workloads serve para ensinar rede, scheduling e arquitetura. Não estamos perseguindo alta disponibilidade completa.

3 Kubernetes em linguagem simples

Kubernetes mantém aplicações em execução a partir de objetos declarativos.

Objeto	Modelo mental	Exemplo na PoC
Cluster	conjunto administrado de nós	NMI ou CIC
Node	máquina que executa workloads	oran-k8s-w02
Pod	um ou mais contêineres co-localizados	kpm-load-watch-*
Deployment	declara réplicas e atualização	deployment/kpm-load-watch
Service	endereço estável para pods	monitoring-grafana
Namespace	escopo lógico de objetos	ricxapp
ConfigMap/Secret	configuração e dado sensível	credenciais e YAML

Spec e status

O manifesto declara a spec, isto é, o estado desejado. Os controladores observam o status e tentam fazer o estado real convergir para o desejado.

```
kubectl get deployment -n ricxapp kpm-load-watch
kubectl describe deployment -n ricxapp kpm-load-watch
kubectl get pods -n ricxapp -o wide
```

NÃO CRIE PODS DIRETAMENTE Deployments administram pods substituíveis. Um pod isolado não oferece a mesma reconciliação e atualização.

4 Namespace não é outro cluster

Namespaces dividem recursos e nomes dentro de uma única API Kubernetes.

No NMI, 5g-core, ran, near-rt-ric, ricxapp, monitoring e nephio-* são namespaces do mesmo cluster de cinco nós. Eles não possuem control planes independentes.

Namespace	Conteúdo
5g-core	Open5GS, MongoDB e WebUI
ran	O-CU, O-DU e srsUE
near-rt-ric	E2Term, E2Mgr, SubMgr, RTMgr, AppMgr e serviços base
ricxapp	r4-simple-mon e kpm-load-watch
monitoring	Prometheus, Grafana, Loki, Alloy e Alertmanager
nephio-* / porch-*	WebUI, Porch e controllers

COMPARAÇÃO Um condomínio pode ter blocos e apartamentos; continua sendo um condomínio. Namespaces organizam o cluster, mas não criam um novo cluster.

```
kubectl get namespaces
kubectl get pods -A
kubectl get pods -n ricxapp
```

5 Kubernetes executa; Nephio entrega

Essa é a distinção mais importante do material.

Kubernetes	Nephio/Porch
agenda pods nos nós	organiza pacotes declarativos
mantém estado desejado	especializa variantes por site
oferece rede e descoberta	publica revisões no Git
reinicia/substitui workloads	automatiza o fluxo de configuração

NÃO É UMA ESCOLHA Usamos Nephio sobre Kubernetes. Sem Kubernetes não há onde executar a xApp. Sem Nephio ainda seria possível aplicar YAML/Helm manualmente.

E Rancher ou OpenShift?

Rancher e OpenShift ajudam a administrar/oferecer a plataforma Kubernetes. Nephio é especializado no ciclo de pacotes e configuração de funções de rede por sites e variantes. Nenhum deles substitui o runtime Kubernetes.

E Helm?

Helm renderiza templates parametrizados. Nephio/Porch governa pacotes, revisões e variantes. Nesta PoC, Helm continua responsável pela pilha base; Nephio foi introduzido de forma incremental para xApps.

6 O que é Open RAN

Open RAN separa funções da RAN e define interfaces mais abertas entre elas.

Função	Responsabilidade	Na PoC
UE	equipamento do usuário	srsUE em software
O-RU	rádio e conversão de sinais	RF substituída por ZMQ
O-DU	funções de rádio de menor camada	OCUDU DU
O-CU	controle e plano de usuário centralizados	OCUDU CU
5G Core	registro, sessão e tráfego	Open5GS
Near-RT RIC	telemetria/controle próximo de tempo real	O-RAN SC RIC

RF SIMULADA ZMQ substitui o enlace de rádio físico. O restante do fluxo — registro, sessão PDU, E2 e KPM — é software real da pilha.

Por que isso é interessante para ensino?

- permite trocar componentes e estudar interfaces;
- separa plano de dados, telemetria e gestão;
- abre espaço para xApps e algoritmos de pesquisa;
- permite começar sem SDR, antena e licenciamento de espectro.

7 UE, SIM, IMSI e IMEI

O assinante, o aparelho e a xApp têm papéis independentes.

Termo	Significado	Quem usa
UE	equipamento do usuário	RAN e Core
SIM	credencial do assinante	UE + Open5GS
IMSI	identidade do assinante	Open5GS autentica
K / OPc	material criptográfico do SIM	UE + Core
IMEI	identidade do aparelho	não substitui o IMSI
PDU Session	sessão de dados 5G	SMF/UPF + UE

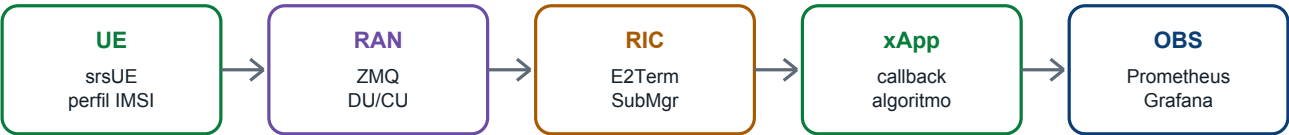
A XAPP NÃO CADASTRA SIM O cadastro ocorre no Open5GS WebUI/MongoDB. A xApp recebe telemetria da RAN pelo E2.

Quando houver rádio e celular reais

Será possível usar um SIM programável, desde que o cartão e o Open5GS compartilhem IMSI, K, OPc/OP e parâmetros compatíveis. Também serão necessários SDR/RU, banda permitida, sincronismo e configuração de RF segura.

8 RIC, xApp, KPI, KPM e RMR

A xApp executa um algoritmo sobre informação recebida do E2 node.



Autenticação do assinante acontece no Open5GS; a xApp não cadastra SIM.

DRB.UEThpDI nasce no O-DU e chega pela cadeia E2SM-KPM/RMR.

O teste estimula tun_srsue e correlaciona o pico no dashboard.

Figura 2 · O caminho didático do assinante até a observabilidade.

Sigla	Definição curta	Exemplo
RIC	controlador inteligente da RAN	Near-RT RIC
xApp	aplicação hospedada pelo RIC	simple-mon
E2	interface RAN ↔ RIC	SCTP/E2AP
KPM	modelo/serviço de medições	E2SM-KPM
KPI	valor de desempenho	DRB.UEThpDI
RMR	mensageria do RIC	indicação 12050

HOJE simple-mon e load-watch monitoram. Elas não enviam ações de controle para a RAN.

9 Três fluxos que não devem ser confundidos

Dados do usuário, telemetria e gestão coexistem, mas são fluxos diferentes.

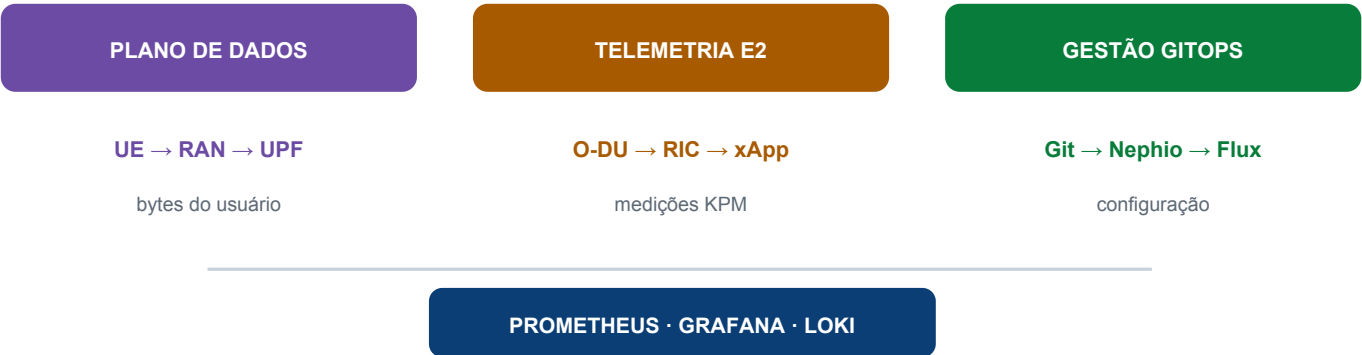


Figura 3 · Dados, telemetria e gestão observados por uma camada comum.

Plano de dados

Transporta o tráfego do usuário: srsUE → RAN → UPF → rede externa.

Telemetria E2

Transporta medições e assinaturas: O-DU → E2Term/SubMgr/RMR → xApp.

Gestão GitOps

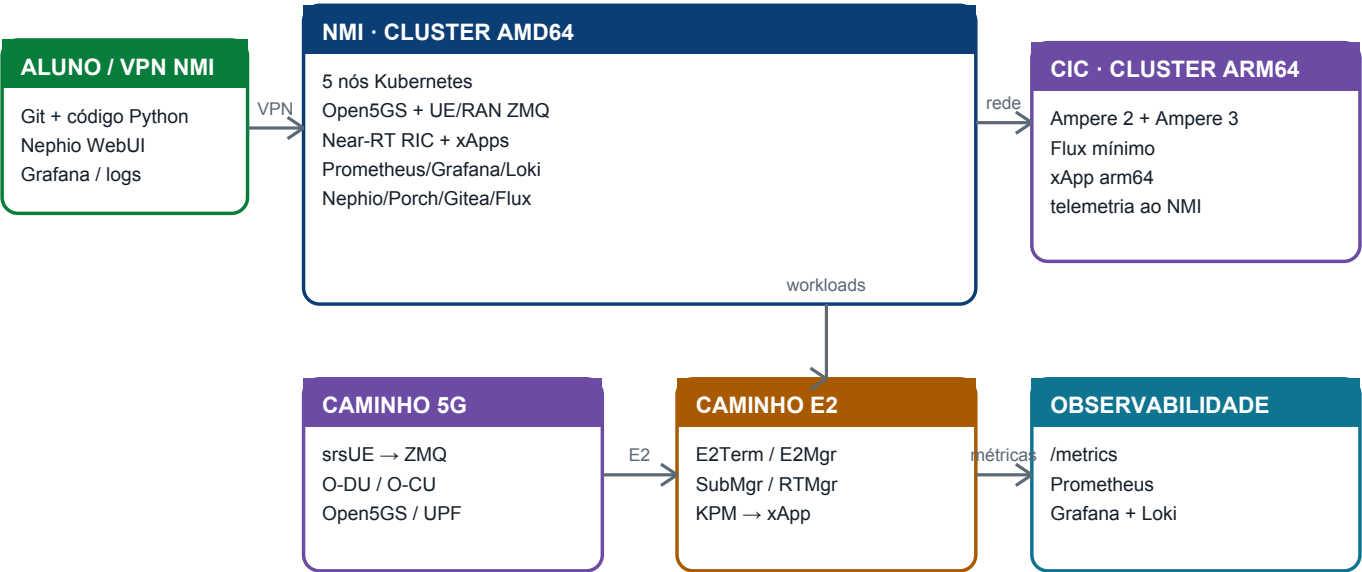
Transporta estado desejado: código/pacote → Porch/Git → Flux → Kubernetes.

OBSERVABILIDADE Prometheus, Grafana e Loki reúnem evidências dos três fluxos sem se tornarem parte do plano de dados 5G.

10 Topologia atual NMI + CIC

A arquitetura implantada combina cinco nós amd64 no NMI e dois nós arm64 no CIC.

Topologia didática multi-site



Gestão: Git → Porch → Gitea → Flux → Kubernetes
Dados: UE → RAN → UPF → Internet
Telemetria: O-DU → E2 → Near-RT RIC → xApp → Grafana

Figura 4 · Topologia lógica multi-site validada em 1º de setembro de 2026.

DECISÃO ARQUITETURAL NMI concentra a pilha 5G, o management plane Nephio e a observabilidade. CIC valida portabilidade ARM sem duplicar toda a pilha.

11 NMI: cinco nós e papéis

O cluster NMI mistura Debian e Ubuntu sem problema; Kubernetes abstrai o sistema operacional desde que os requisitos sejam compatíveis.

Nó	SO	Papel principal	Workloads
oran-k8s-01	Debian 12	control plane + telecom	Open5GS, RAN, RIC base
oran-k8s-w01	Debian 12	RIC distribuído	E2Term, RTMgr
oran-k8s-w02	Debian 12	xApps	simple-mon, load-watch
nmi-srv03	Ubuntu 24.04	observabilidade	Prometheus, Grafana, Loki
nephio-k8s-w01	Ubuntu 22.04	gestão GitOps	Porch, Gitea, Flux, WebUI

SNAPSHOT Todos os cinco nós estavam Ready, amd64 e em Kubernetes v1.30.14 na consulta de 1º de setembro de 2026.

```
kubectl get nodes -o wide
kubectl get pods -A -o wide
```

12 CIC: cluster ARM mínimo

Duas VMs Ampere formam um cluster separado para experimentar workloads arm64.

VM	Host	Endereço	Recursos	Papel
120 · cic-k8s-cp01	Ampere 2	192.168.0.210	8 vCPU · 16 GiB · 100 GB	control plane
121 · cic-k8s-w01	Ampere 3	192.168.0.211	8 vCPU · 16 GiB · 100 GB	worker/xApp

- Kubernetes v1.32.13 arm64;
- Flannel, Multus, OVS-CNI e local-path;
- Flux mínimo no CIC; Nephio/Porch continuam no NMI;
- r4-simple-mon-cic executa nativamente como aarch64;
- telemetria do CIC é centralizada no Grafana NMI.

POR QUE OUTRO CLUSTER? É outro site e outra arquitetura. Um cluster Kubernetes multiarch seria possível, mas não representaria tão bem a separação física NMI/CIC nem simplificaria a conectividade dos dois laboratórios.

13 Rede: Flannel, Multus e OVS

A PoC usa uma rede de cluster e redes adicionais de telecom.

Tecnologia	Resolve	Na PoC
Flannel	pod ↔ pod entre nós	overlay VXLAN padrão
Service/DNS	descoberta estável de pods	*.svc.cluster.local
Multus	mais de uma interface por pod	anexa redes de telecom
OVS-CNI	bridges e conectividade L2	n2br, f1cbr, e2br
pfSense	roteamento/firewall entre redes	.71, .72 e CIC

DOIS PROBLEMAS Flannel mantém a comunicação Kubernetes simples; Multus/OVS preserva interfaces e endereços específicos de N2, F1-C e E2.

Por que SCTP aparece tanto?

NGAP/N2 e E2AP usam SCTP. Por isso, firewall, NAT e caminhos multi-host precisam ser validados com atenção; HTTP funcionando não garante que SCTP esteja liberado.

14 Distribuição dos workloads

A colocação de pods foi escolhida para mostrar o laboratório distribuído e preservar interfaces sensíveis.

Domínio	Nó predominante	Motivo
5G Core	oran-k8s-01	baseline estável e acesso às redes
RAN	oran-k8s-01	ZMQ e interfaces de telecom
RIC base	oran-k8s-01	serviços centrais
E2Term / RTMgr	oran-k8s-w01	provar RIC multi-host
xApps	oran-k8s-w02	isolar algoritmos
Observabilidade	nmi-srv03	CPU/RAM e armazenamento
Nephio	nephio-k8s-w01	isolar a gestão em outro worker

NAMESPACE NÃO FIXA NÓ A colocação resulta de affinity/nodeSelector/taints e do scheduler. O namespace organiza; ele não determina sozinho onde o pod roda.

15 Observabilidade e acesso

O NMI centraliza métricas, dashboards, logs curtos e alertas para toda a PoC.

Componente	Função	Onde roda
Prometheus	coleta e armazena métricas	nmi-srv03
Grafana	dashboards e correlação	nmi-srv03
Loki	logs com retenção curta	nmi-srv03
Alloy	coleta logs nos nós	DaemonSet
Alertmanager	agrupa alertas	nmi-srv03

LOGS LIMITADOS O laboratório evita loops de log e retenção longa. O objetivo é diagnóstico, não arquivamento de produção.

Acesso

As interfaces são privadas e acessíveis pela VPN institucional do NMI. Credenciais ficam em Secrets/cofre ou são fornecidas pelo responsável; não devem ser publicadas no repositório ou neste guia.

Interface	URL pela VPN
Open5GS WebUI	http://192.168.72.10:30454
Nephio WebUI	http://192.168.71.30:30707/config-as-data
Grafana Overview	http://192.168.72.10:30300/d/oran-overview/o-ran-stack-overview
Grafana Multi-site	http://192.168.72.10:30300/d/oran-multisite/cic-arm-lab
Grafana Logs	http://192.168.72.10:30300/d/oran-logs/o-ran-kubernetes-logs
Alertmanager	http://192.168.72.10:30301

16 O repositório como mapa

O código versionado é a fonte de conhecimento e reprodução da bancada.

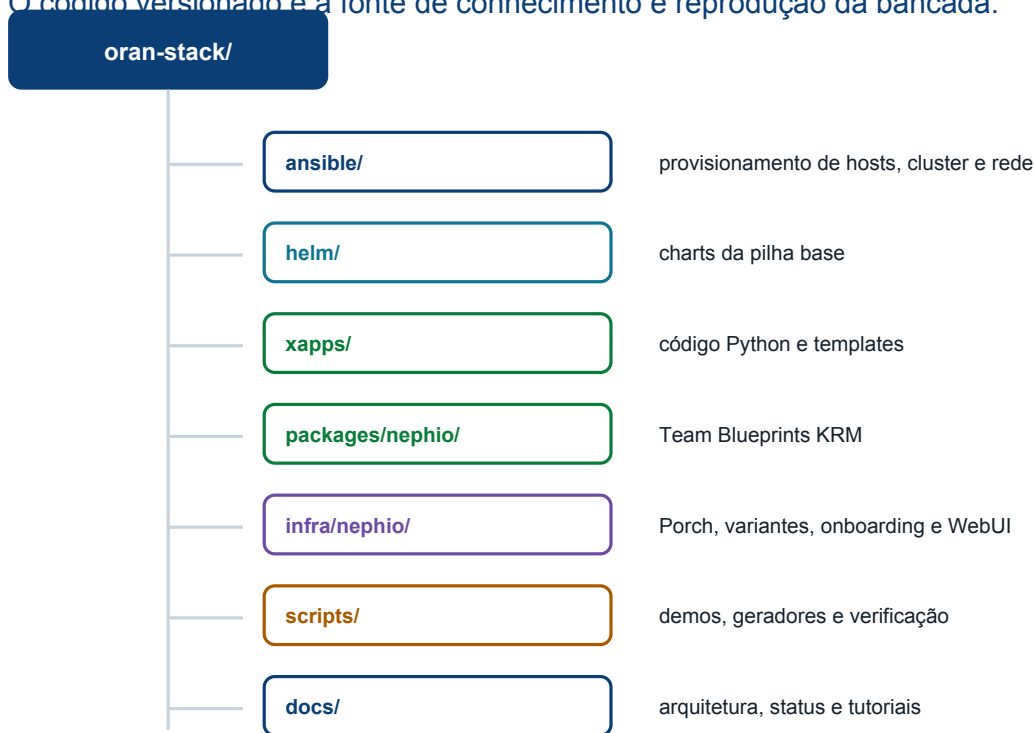


Figura 5 · Diretórios principais e sua responsabilidade.

LEITURA RECOMENDADA README.md explica a entrada; docs/STATUS.md registra fatos e resultados; os inventários e charts materializam a arquitetura.

17 Ansible: preparar a base

Ansible automatiza máquinas, Kubernetes, rede, imagens e a implantação inicial.

- inventários descrevem hosts e variáveis;
- roles instalam container runtime, Kubernetes e plugins;
- playbooks criam/juntam os nós ao cluster;
- tarefas configuram Multus, OVS e bridges;
- playbooks de deploy instalam charts Helm da pilha base.

```
ansible/  
├─ inventories/  
├─ playbooks/  
└─ roles/
```

LIMITAÇÃO INTENCIONAL Nephio não substituiu Ansible no provisionamento de Proxmox, VMs ou Kubernetes. O caminho de bootstrap continua independente.

18 Helm: empacotar a pilha base

Os charts organizam recursos Kubernetes parametrizados.

Chart	Conteúdo
helm/5g-core	Open5GS, MongoDB, WebUI e Services
helm/ran	O-CU, O-DU, srsUE e redes adicionais
helm/near-rt-ric	plataforma RIC e rotas
helm/xapps	caminho histórico/base das xApps
helm/monitoring	Prometheus, Grafana, Loki e alertas

Helm transforma values + templates em YAML. O Kubernetes continua sendo quem executa e reconcilia os objetos resultantes.

MIGRAÇÃO INCREMENTAL As xApps de laboratório passaram a ter entrega por Nephio/Flux, mas a pilha base permanece reproduzível por Helm/Ansible.

19 Código das xApps

A base Python encapsula integração com o RIC para que o aluno foque no algoritmo.

Arquivo/diretório	Responsabilidade
xapps/python/xAppBase.py	registro, assinatura, RMR, decode e métricas
xapps/python/kpm_mon_xapp.py	simple-mon de referência
xapps/python/kpm_load_watch_xapp.py	média móvel e estados
xapps/templates/kpm-monitor	scaffold para nova xApp
tests/	testes automatizados

CONTRATO Uma xApp deve registrar-se, criar uma assinatura, receber RMR, decodificar KPM, limitar logs e expor /metrics.

20 Pacotes Nephio

Packages e infra/nephio registram o ciclo declarativo das xApps.

Caminho	Conteúdo
packages/nephio/r4-simple-mon	blueprint reutilizável
packages/nephio/kpm-load-watch	blueprint da xApp do aluno
infra/nephio/blueprints	publicação e PackageVariants
infra/nephio/nmi-onboarding	RBAC e Flux do NMI
infra/nephio/cic-onboarding	entrega mínima ao CIC
infra/nephio/webui-compatible	WebUI e identidade visual UnB

OBJETOS VIVOS WorkloadClusters nmi/cic e PackageVariants simple-mon-nmi-v5, simple-mon-cic-arm64 e kpm-load-watch-nmi estavam presentes no snapshot.

21 Scripts, documentação e evidência

Uma boa PoC precisa ser demonstrável e auditável.

Artefato	Uso
scripts/demo-kpm.sh	gera tráfego e observa KPM
scripts/demo-load-watch.sh	prova estados idle/active/busy
scripts/demo-kpm-multisite.sh	compara NMI e CIC
scripts/demo-ue-lab.sh	troca perfis UE sequenciais
scripts/new-kpm-xapp.sh	gera uma nova xApp
docs/STATUS.md	linha do tempo, fatos e limitações
docs/STUDENT_XAPP_LAB.md	laboratório do aluno
docs/UE_SIMULATION_LAB.md	perfis e limites ZMQ

CIÊNCIA REPRODUZÍVEL O script define o estímulo; Prometheus/Grafana registram a resposta; Git registra a configuração e o código.

22 Plano de dados de ponta a ponta

O download do UE atravessa RAN e Core; o RIC não carrega os bytes do usuário.

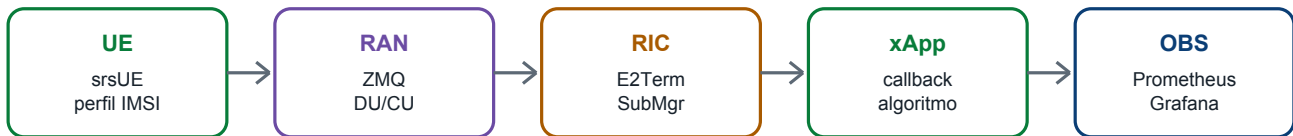
Etapa	Evento
1	srsUE estabelece enlace ZMQ com a RAN
2	UE executa registro NAS pelo AMF
3	SMF cria a sessão PDU e programa o UPF
4	tun_srsue recebe endereço do pool 10.45.0.0/16
5	requisição HTTP sai pelo GTP-U/UPF e recebe retorno
6	O-DU mede throughput e gera KPM em paralelo

```
ip addr show tun_srsue
curl --interface tun_srsue -I http://example.com/
sudo ./scripts/demo-kpm.sh
```

PROVA O laboratório validou registro 5G SA, sessão PDU, egress com HTTP 200 e transferência limitada de 50 MB.

23 Caminho E2/KPM

A xApp só é funcional quando a indicação percorre toda a cadeia.



Autenticação do assinante acontece no Open5GS; a xApp não cadastra SIM.

DRB.UETHPDI nasce no O-DU e chega pela cadeia E2SM-KPM/RMR.

O teste estimula tun_srsue e correlaciona o pico no dashboard.

Critérios de aceite

- E2 node aparece CONNECTED no E2Mgr;
- AppMgr lista a xApp e seu endpoint RMR;
- SubMgr mantém uma assinatura ativa;
- contadores RMR e KPM aumentam;
- o valor DRB.UETHPDI muda com o estímulo;
- Prometheus descobre o ServiceMonitor/Service;
- Grafana mostra a série separada pelo label da xApp.

RUNNING NÃO BASTA Um pod Running prova apenas que o processo iniciou. A aceitação exige estímulo conhecido e resposta mensurável.

24 Nephio e GitOps de ponta a ponta

O pacote é preparado por intenção e entregue por reconciliação.



O aluno produz código, teste, imagem e blueprint.

A infraestrutura revisa digest, RBAC, portas e variante do site.

Nephio publica configuração; Flux entrega; Kubernetes executa.

Figura 6 · Pipeline declarativo da xApp na PoC.

O papel de cada componente

- Team Blueprint: modelo reutilizável sem detalhes de um site;
- PackageVariant: intenção que especializa NMI ou CIC;
- Porch: orquestra revisões e transformação de pacotes;
- Gitea: armazena o pacote publicado;
- Flux: reconcilia o Git no workload cluster;
- Kubernetes: executa Deployment, Service e demais recursos.

ROLLBACK PARA FRENTE Em vez de apagar histórico, publicamos uma revisão posterior que restaura a configuração desejada.

25 Comparação objetiva das ferramentas

A tabela abaixo evita sobreposição de responsabilidades.

Ferramenta	Pergunta	Papel
Proxmox	onde criar as VMs?	virtualização
Ansible	como preparar hosts/cluster?	provisionamento
Kubernetes	onde executar pods?	runtime/reconciliação
Helm	como parametrizar YAML?	templating/pacote
Nephio/Porch	qual variante vai para cada site?	ciclo de pacotes
Flux	como aplicar o Git?	reconciliador GitOps
Rancher/OpenShift	como administrar a plataforma?	não usados
RIC	como interagir com a RAN via E2?	plataforma telecom

ATALHO Helm empacota templates; Nephio governa variantes; Flux sincroniza Git; Kubernetes executa; o RIC conversa com a RAN.

26 Resultados comprovados

A PoC possui uma cadeia de evidências, não apenas pods verdes.

Evidência	Resultado
Cluster NMI	5/5 nós Ready, Kubernetes v1.30.14
Cluster CIC	2 nós ARM64 funcionais
Open5GS	três perfis de assinante e sessões testadas
RAN/E2	DU conectado ao Near-RT RIC
simple-mon	KPM no NMI e no CIC
load-watch	média móvel de 5 amostras e estados
Tráfego	50 MB; pico de 28.841 kbps
GitOps	3 Flux Kustomizations Ready
Nephio	2 WorkloadClusters e 3 PackageVariants

CICLO OBSERVADO idle → active → busy → active → idle durante o teste controlado de tráfego.

27 Limitações atuais e agenda de pesquisa

As limitações são conhecidas e úteis para formular os próximos experimentos.

Hoje	Consequência	Próximo experimento
um peer ZMQ	um perfil UE ativo por vez	múltiplos UEs/RUs ou SDR
KPM agregado	sem atribuição direta ao IMSI	Report Style/per-UE + correlação
xApps monitoram	sem ação de controle	E2SM-RC + guardrails
management no NMI	dependência de um site	estudo de placement/latência
PoC sem HA	falhas interrompem partes	automação de recuperação didática

DRB.UETpDI STYLE 1 Com noLabel=true, a medição é agregada no O-DU/célula. Ela prova carga de rádio, mas não diz que o valor pertence ao IMSI ...001, ...002 ou ...003.

Para medir por UE será preciso suporte do E2 node a outro Report Style/label e correlacionar identificadores de RAN com a identidade do assinante no Core.

28 Pré-requisitos para reproduzir

A reprodução começa com acesso e uma baseline saudável.

- VPN institucional do NMI ativa;
- conta SSH de laboratório e chave pública autorizada;
- clone atualizado do repositório;
- kubectl e KUBECONFIG apropriado;
- credenciais fornecidas separadamente/Secrets;
- permissão para usar os scripts de demo;
- janela de teste sem outro aluno alterando RAN/RIC.

NÃO PUBLIQUE SEGREDOS IPs privados podem aparecer na documentação do laboratório; senhas, tokens, chaves privadas, K e OPc não devem aparecer em commit ou PDF público.

Clone

```
git clone https://github.com/lucasrodri/oran-stack.git
cd oran-stack
git status
git log -5 --oneline
```

29 Pré-voo do cluster

Valide a infraestrutura antes de culpar a nova xApp.

```
kubect1 get nodes -o wide
kubect1 get namespaces
kubect1 get pods -A -o wide
kubect1 get svc -A
kubect1 get events -A --sort-by=.lastTimestamp | tail -n 40
```

Critério mínimo

- todos os nós esperados estão Ready;
- pods de 5g-core, ran, near-rt-ric e monitoring estão Ready;
- nenhum CrashLoopBackOff cresce continuamente;
- Services NodePort estão presentes;
- não há evento recente de falta de disco/memória.

PARE Se o baseline estiver quebrado, registre a evidência e trate a infraestrutura primeiro. Não modifique o algoritmo para compensar uma RAN desconectada.

30 Pré-voo do Core, RAN e RIC

O experimento exige UE, sessão PDU e E2 node conectado.

```
kubect1 -n 5g-core get pods
kubect1 -n ran get pods -o wide
kubect1 -n near-rt-ric get pods -o wide
curl -fsS http://127.0.0.1:30380/v1/nodeb/states
```

Sinais de saúde

- Open5GS NFs e MongoDB estão Ready;
- O-CU, O-DU e srsUE estão Ready;
- UE conclui RRC e PDU Session;
- E2 node gnb*_ aparece CONNECTED;
- E2Term, E2Mgr, SubMgr, RTMgr e AppMgr estão Ready.

DEPENDÊNCIAS Uma xApp KPM depende de RAN + E2 + RIC. A WebUI do Open5GS, por sua vez, cuida dos assinantes e não prova a saúde do E2.

31 Pré-voo do Nephio e Flux

Nephio e Flux devem mostrar os objetos que representam NMI e CIC.

```
kubectl get workloadclusters.infra.nephio.org
kubectl get packagevariants.config.porch.kpt.dev
kubectl get kustomizations.kustomize.toolkit.fluxcd.io -A
```

Objeto esperado	Estado de referência
WorkloadCluster/nmi	presente
WorkloadCluster/cic	presente
PackageVariant/kpm-load-watch-nmi	presente
PackageVariant/simple-mon-nmi-v5	presente
PackageVariant/simple-mon-cic-arm64	presente
Flux Kustomizations	Ready=True

INTERPRETAÇÃO PackageVariant presente não significa pod saudável. Confirme também o Flux e o Deployment no workload cluster.

32 Comandos kubectl essenciais

Os comandos abaixo cobrem 80% do diagnóstico inicial.

```
# inventário
kubectl get nodes -o wide
kubectl get pods -A -o wide
kubectl get svc -A

# detalhes
kubectl describe pod -n <namespace> <pod>
kubectl get deployment -n <namespace> <nome> -o yaml

# logs
kubectl logs -n <namespace> <pod> --tail=100
kubectl logs -n <namespace> deployment/<nome> --tail=100
kubectl logs -n <namespace> <pod> --previous --tail=100

# acompanhar
kubectl get pods -n ricxapp -w
kubectl rollout status -n ricxapp deployment/<nome> --timeout=180s
```

CUIDADO get, describe e logs são consultas. apply, delete, rollout restart e scale alteram estado; use somente quando o roteiro autorizar.

33 Acessar as interfaces

Cada WebUI responde a uma pergunta diferente.

Interface	Pergunta
Open5GS WebUI	quais assinantes/perfis existem?
Nephio WebUI	quais repositórios e pacotes estão publicados?
Grafana Overview	Core, RAN, RIC, xApps e nós estão saudáveis?
Grafana Multi-site	NMI e CIC recebem a mesma telemetria?
Grafana Logs	qual componente registrou erro?
Alertmanager	qual condição ultrapassou uma regra?

VPN Use os endereços privados pela VPN NMI.

34 Troubleshooting por cadeia

Diagnostique do estímulo para a evidência, sem reiniciar tudo.

Sintoma	Primeiras verificações
UE não registra	perfil Open5GS, K/OPc, RAN, AMF, RRC/NAS
sem tun_srsue	PDU Session, SMF/UPF, pool e DNN
sem egress	rota, NAT UPF, pfSense e interface correta
E2 desconectado	E2Term, SCTP, e2br, node ID e RTMgr
xApp sem assinatura	AppMgr, SubMgr, RAN function e período
KPM zero	tráfego real, métrica implementada e labels
Grafana No data	ServiceMonitor, target Prometheus e consulta
Flux não Ready	GitRepository, path, RBAC e YAML inválido

MÉTODO Mude uma variável por vez, limite logs, preserve a evidência anterior e registre a hipótese no relatório.

35 Começar uma nova xApp pela pergunta

Um experimento bom nasce de uma hipótese, não de um Deployment.

Exemplo de pergunta

“Uma média móvel de DRB.UETpDI consegue classificar a carga da RAN em idle, active e busy durante um estímulo controlado?”

Elemento	Definição
Entrada	DRB.UETpDI a cada 3000 ms
Transformação	média móvel de cinco amostras
Saída	estado 0/1/2 + valor médio
Estímulo	transferência de 50 MB pela tun_srsue
Aceite	ciclo idle → active → busy → active → idle

ANTES DE PROGRAMAR Defina métrica, período, janela, limiares, estímulo e evidência. Isso evita uma xApp que “roda” mas não responde a uma pergunta.

36 Gerar o scaffold

O gerador cria código, pacote e objetos de onboarding coerentes.

```
git switch main
git pull --ff-only
git switch -c aluno/kpm-latency-lab

./scripts/new-kpm-xapp.sh kpm-latency-lab DRB.UEThpD1 2000
```

Artefato	Finalidade
xapps/python/kpm_latency_lab_xapp.py	algoritmo
packages/nephio/kpm-latency-lab	Team Blueprint
infra/nephio/blueprints/*variant.yaml	variante NMI
infra/nephio/nmi-onboarding/*rbac.yaml	permissões
infra/nephio/nmi-onboarding/*flux-sync.yaml	reconciliação

NOME Use letras minúsculas, números e hífens. O gerador não deve sobrescrever uma xApp existente.

37 Programar o callback

xAppBase abstrai a plataforma; o aluno implementa a transformação da medição.

```
def indication_callback(self, e2_agent_id, subscription_id,
                        indication_header, indication_message):
    measurements = self.e2sm_kpm.extract_meas_data(indication_message)
    value = measurements.get("measData", {}).get("DRB.UEThpD1", 0)

    self.window.append(float(value or 0))
    average = sum(self.window) / len(self.window)
    state = "busy" if average >= 20000 else "active" if average >= 1000 else
           "idle"
    self.export_state(average, state)
```

Boas práticas

- callback rápido e não bloqueante;
- validação de valor ausente/não numérico;
- logs apenas em transição ou amostrados;
- métricas com labels controlados;
- estado interno pequeno e testável;
- sem credenciais no código.

38 Testar antes do cluster

A maioria dos erros simples deve falhar na máquina do aluno ou na CI.

```
python3 -m py_compile xapps/python/kpm_latency_lab_xapp.py
python3 -m unittest discover -s tests -v
bash -n scripts/new-kpm-xapp.sh
git diff --check

rg 'REPLACE_WITH_MULTIARCH_IMAGE_DIGEST|:latest'    packages/nephio/kpm-latency-lab
infra/nephio/blueprints/kpm-latency-lab-nmi-variant.yaml
```

ACEITE LOCAL Código compila, testes passam, shell é válido, não há whitespace quebrado e nenhum marcador de imagem permanece antes da publicação.

Teste de unidade útil

Alimente o algoritmo com uma sequência sintética de valores e verifique transições e média. Esse teste independe do RIC e reduz o tempo de diagnóstico no cluster.

39 Publicar uma imagem multiarch

A mesma referência imutável deve atender amd64 e arm64.

A CI constrói um índice OCI com linux/amd64 e linux/arm64. O pacote deve usar o digest do índice, não :latest e não o digest de uma única plataforma.

```
git add xapps/python/kpm_latency_lab_xapp.py packages/nephio/kpm-latency-lab
infra/nephio/blueprints/kpm-latency-lab-nmi-variant.yaml infra/nephio/nmi-
onboarding/kpm-latency-lab-*.yaml

git commit -m "feat(xapp): add kpm latency lab monitor"
git push -u origin aluno/kpm-latency-lab
```

IMUTABILIDADE Digest permite provar exatamente qual conjunto de bytes foi executado nos dois sites.

Referência esperada

```
ghcr.io/lucasrodri/oran-stack/oran-xapps@sha256:<digest-do-índice>
```

40 Publicar blueprint e variante

O Team Blueprint vira um pacote específico do NMI por meio do PackageVariant.



O aluno produz código, teste, imagem e blueprint.

A infraestrutura revisa digest, RBAC, portas e variante do site.

Nephio publica configuração; Flux entrega; Kubernetes executa.

```
export KUBECONFIG=/etc/nephio/nmi-admin.conf

sudo -E ./infra/nephio/blueprints/publish-xapp-blueprint.sh kpm-latency-lab
packages/nephio/kpm-latency-lab v1

sudo -E kubectl apply -f infra/nephio/blueprints/kpm-latency-lab-nmi-variant.yaml

sudo -E ./infra/nephio/blueprints/approve-packagevariant.sh kpm-latency-lab-nmi
```

O QUE O NEPHIO FEZ Publicou e especializou configuração. Ele ainda não iniciou o processo da xApp; isso acontece depois da reconciliação no Kubernetes.

41 Entregar pelo Flux e aceitar

A reconciliação deve resultar em workload funcional, não apenas em Git atualizado.

```
sudo -E kubectl apply -f infra/nephio/nmi-onboarding/kpm-latency-lab-rbac.yaml
sudo -E kubectl apply -f infra/nephio/nmi-onboarding/kpm-latency-lab-flux-sync.yaml

sudo -E kubectl -n flux-system wait kustomization/kpm-latency-lab --for=condition=Ready
--timeout=300s

kubectl -n rickapp rollout status deployment/kpm-latency-lab --timeout=180s
```

Aceite da plataforma

- Flux Ready=True e revisão Git esperada;
- Deployment disponível;
- pod Ready sem restart crescente;
- Service e /metrics acessíveis;
- AppMgr e SubMgr reconhecem a xApp;
- contadores RMR/KPM aumentam.

42 Estimular, observar, atualizar e reverter

O experimento termina com evidência e histórico.

```
sudo env KUBECONFIG=/etc/kubernetes/admin.conf XAPP_DEPLOYMENT=kpm-latency-lab
./scripts/demo-kpm.sh

# referência didática
sudo env KUBECONFIG=/etc/kubernetes/admin.conf ./scripts/demo-load-watch.sh
```

O que observar

- assinatura ativa = 1;
- taxa de indicações > 0;
- pico de DRB.UETpDI durante a transferência;
- retorno próximo de zero depois do estímulo;
- estado/transições coerentes com os limiares.

Update e rollback

Altere um parâmetro ou versão do blueprint, publique a revisão seguinte, espere o Flux reconciliar e repita o teste. Para reverter, publique uma nova revisão que restaure os valores anteriores.

RELATÓRIO Registre hipótese, versão/digest, parâmetros, horário, estímulo, gráfico, resultado e limitações. Isso transforma uma demo em experimento.

A Glossário rápido

Termo	Definição
5G Core	núcleo que registra assinantes e transporta sessões
AMF	função de acesso e mobilidade
SMF	função de gestão de sessão
UPF	plano de usuário e egress
RAN	rede de acesso por rádio
O-DU / O-CU	funções distribuída e centralizada da RAN
Near-RT RIC	plataforma de controle/telemetria da RAN
xApp	aplicação executada junto ao Near-RT RIC
E2	interface entre E2 node e Near-RT RIC
E2SM-KPM	modelo de serviço para medições
KPI	indicador de desempenho
RMR	mensageria interna do RIC
GitOps	reconciliação de estado desejado a partir do Git
Porch	orquestração de pacotes kpt/KRM
PackageVariant	especialização declarativa de um pacote
Flux	reconciliador GitOps

B Folha de consulta kubectl

```
kubectl get nodes -o wide
kubectl get ns
kubectl get pods -A -o wide
kubectl get svc -A
kubectl get deploy -A
kubectl get events -A --sort-by=.lastTimestamp

kubectl describe pod -n <ns> <pod>
kubectl logs -n <ns> <pod> --tail=100
kubectl logs -n <ns> <pod> --previous --tail=100
kubectl exec -n <ns> -it <pod> -- /bin/sh

kubectl rollout status -n <ns> deployment/<nome>
kubectl rollout history -n <ns> deployment/<nome>

kubectl get workloadclusters.infra.nephio.org
kubectl get packagevariants.config.porch.kpt.dev
kubectl get kustomizations.kustomize.toolkit.fluxcd.io -A
```

CONSULTA ANTES DE MUDANÇA Use get/describe/logs para localizar a camada da falha. Não reinicie toda a pilha como primeira resposta.

C Roteiro de demonstração em 10 minutos

Tempo	Ação	Mensagem
00–02	mostrar topologia e nós	5 nós NMI + 2 nós ARM
02–04	abrir Open5GS	assinantes e sessão são do Core
04–06	abrir Nephio	blueprint + variantes NMI/CIC
06–08	executar estímulo UE	50 MB atravessam a rede 5G
08–10	abrir Grafana	KPM e estados comprovam o efeito

FRASE FINAL Código no Git virou pacote por site, pod em duas arquiteturas e evidência de rádio no Grafana.

D Checklist do relatório do aluno

- ☐ pergunta de pesquisa e hipótese;
- ☐ arquitetura e versão do cluster;
- ☐ nome da xApp, branch, commit e digest;
- ☐ métrica, período, janela e limiares;
- ☐ PackageVariant e revisão Flux;
- ☐ perfil UE e estímulo aplicado;
- ☐ gráfico antes/durante/depois;
- ☐ logs apenas dos eventos relevantes;
- ☐ resultado esperado versus observado;
- ☐ limitações e ameaça à validade;
- ☐ proposta do próximo experimento.

CRITÉRIO O leitor deve conseguir repetir o estímulo e verificar se obtém a mesma tendência, mesmo que os valores exatos variem.

E Referências e fontes

Este guia combina documentação oficial com o estado observado e os arquivos versionados da PoC.

Documentação oficial

- Kubernetes Concepts: <https://kubernetes.io/docs/concepts/>
- Kubernetes Objects: <https://kubernetes.io/docs/concepts/overview/working-with-objects/>
- Kubernetes Namespaces: <https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/>
- Kubernetes Pods: <https://kubernetes.io/docs/concepts/workloads/pods/>
- Kubernetes Services: <https://kubernetes.io/docs/concepts/services-networking/service/>
- Nephio documentation: <https://docs.nephio.org/docs/>
- Nephio Porch: <https://docs.nephio.org/docs/porch/>
- O-RAN Alliance: <https://www.o-ran.org/>

Fontes do repositório

- README.md;
- docs/STATUS.md;
- docs/NMI_DEPLOYMENT.md;
- docs/CIC_ARM_CLUSTER.md;
- docs/OBSERVABILITY.md;
- docs/STUDENT_XAPP_LAB.md;
- docs/UE_SIMULATION_LAB.md;
- packages/nephio/ e infra/nephio/;
- snapshot kubect! do cluster NMI em 1º de setembro de 2026.

VERSÃO O ambiente é uma bancada viva. Antes de uma aula, confirme STATUS.md, os nós Ready e as revisões Flux em uso.